SOMAIYA VIDYAVIHAR UNIVERSITY
K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)

Somaiya TRUST

Module 4 - Planning

Dr. Nirmala Shinde Baloorkar
Assistant Professor
Department of Computer Engineering, KJSSE

Why not searching?

- Overwhelmed by irrelevant actions
- Difficult to find a good heuristic function
- Lacks autonomy
- Can’t exploit problem decomposition

Why planning?

- Can use domain-independent heuristic
- Can work on sub-goals independently
- Can work out own plans
- Can be autonomous
- Synthesize goal directed behaviors
- Can handle causal dependencies

Planning

- A **planning problem** is the process of finding a sequence (or partially ordered set) of actions that transforms an **initial state** into a **goal state**.
- Planning = Finding an **action sequence** that satisfies goal conditions from a given initial state.

Components of Planning Problem

- Initial State
- Goal State
- Actions

A planning problem

- Goal: Have a birthday party
- Current situation: Agent is at home, Has flour, Does not have butter, Does not have sugar.
- To Do:

Invite friends, Buy butter, Buy sugar, Buy balloons, Decorate house, Bake cake, ...

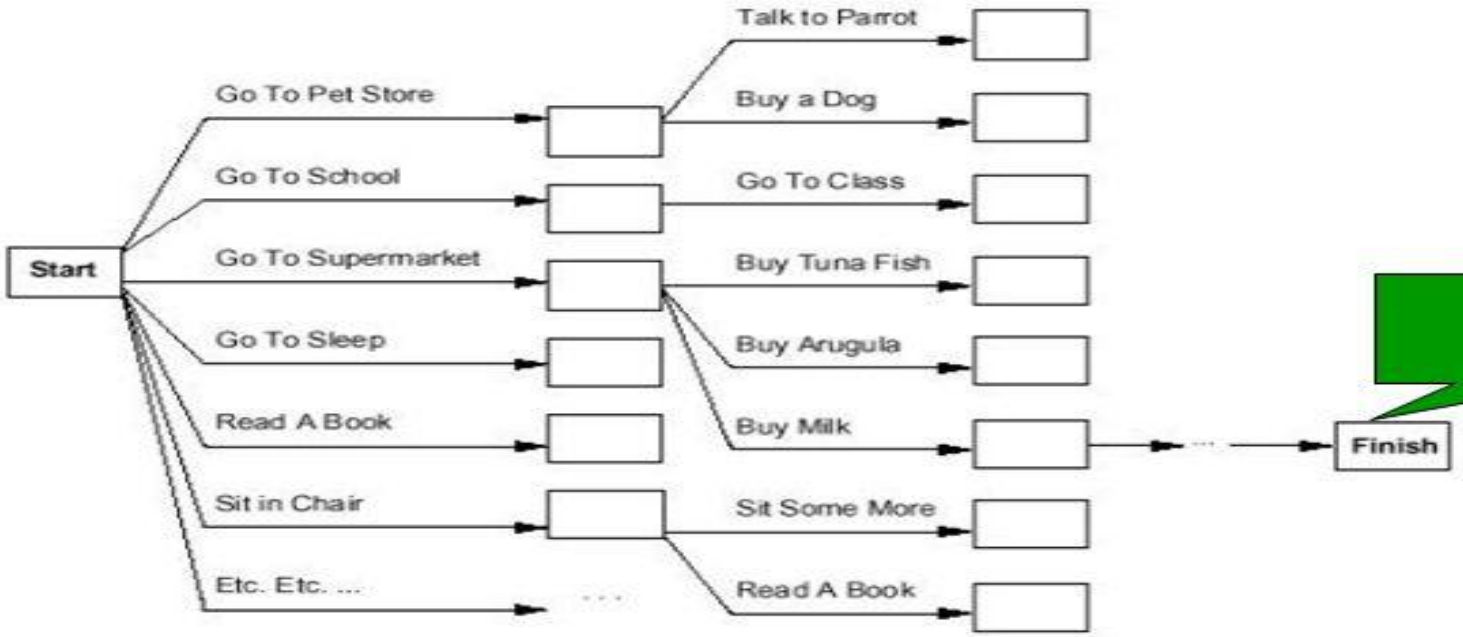
Ref: Prof Sarkar's slides, IIT KGP

Possible actions

- invite friends,
- go out and buy butter,
- go out and buy sugar,
- buy balloons,
- decorate the house,
- bake the cake and so on.

Difficulty using standard search algorithms

Standard search algorithms seems to fail since the goal test is inadequate.



What is "finish"?

Courtesy: http://images.slideplayer.com/34/8320104/slides/slide_3.jpg

Language for planning problem

- STRIPS - STanford Research Institute's Problem Solver, Stanford, 1971
- ADL – Action Description Language, IBM 1987
- PDDL
  - Developed for **AI planning competitions** (late 1990s).
  - Inspired by **STRIPS & ADL**, but **standardized for multiple planners**.

- Expressive enough to describe a wide variety of problems, but restrictive enough to allow efficient algorithms to operate over it

STRIPS (Stanford Research Institute Problem Solver)

- One of the earliest AI planning languages (1971).
- Uses **preconditions, actions, and effects** to describe planning problems.
- **Example Syntax:**

Action: MOVE(A, B)

Preconditions: AT(A)

Effects: ¬AT(A), AT(B)

- **Limitations:** Cannot handle conditional effects or numeric values
- Cannot express conditional effects or quantification.
- Limited expressiveness compared to modern languages.

ADL (Action Description Language)

- **Enhancement of STRIPS** with richer expressiveness.
- Introduced **conditional effects, disjunctions, and quantified variables** in preconditions/effects.
- **Example Syntax:**

Action: PICK_UP(x)

Preconditions: CLEAR(x) ∧ ON_TABLE(x)

Effects: HOLDING(x) ∧ ¬ON_TABLE(x)

- **Benefit:** Supports more complex AI planning tasks.

Reading: Other planning languages

Language	Features & Use Cases
HTN (Hierarchical Task Networks)	Decomposes tasks into subtasks (e.g., SHOP, SHOP2). Used in robotics & gaming.
Golog	Based on situation calculus; great for reasoning over dynamic environments.
RDDL (Relational Dynamic Influence Diagram Language)	Extends PDDL with probabilistic effects. Used in MDP & POMDP planning.
PPDDL (Probabilistic PDDL)	Variant of PDDL that includes uncertainty & probabilistic actions.
ANML (Action Notation Modeling Language)	Used in NASA's planning tools; supports concurrency & time windows.
NDDL (New Domain Definition Language)	Used in NASA's EUROPA planner for space missions.
XSB Prolog	Logic programming-based planning, useful in AI reasoning.
Planning with TDL (Task Description Language)	Used in robotics & multi-agent systems.
ASP (Answer Set Programming)	Logic-based planning used in constraint-solving and knowledge-based AI.

Problem description

- Representation of states
- Representation of goals
- Representation of actions

Representation of states

- Decompose the world into logical conditions
- Represent a state as a conjunction of positive literals.

E.g., Poor ∩ Unknown

E.g. At(Plane1, Melbourne) ∩ At(Plane2, Sydney)

- Literals in first-order state descriptions must be ground and function-free.

i.e. At(x, y) or At(Father(Fred), Sydney) not allowed.

Representation of goals

- **A goal is a partially specified state, represented as a conjunction** of positive ground literals.
- E.g.Rich ∩ Famous or At(P2, Tahiti).
- A propositional state *s* ***satisfies a goal g if s contains all the atoms in g (and possibly others).***

e.g. the state (Rich ∩ Famous ∩ Miserable) satisfies the goal (Rich ∩ Famous).

Representation of actions

- An action is specified in terms of the preconditions
- The action name and parameter list-

e.g. Fly(p, from, to)-serves to identify the action.

- The precondition is a conjunction of function-free positive literals
- Any variables in the precondition must also appear in the action's parameter list.
- The effect is a conjunction of function-free literals

STRIPS solution for transportation of air cargo between airports.

$$
\begin{aligned}
&Init(At(C_1, SFO) \wedge At(C_2, JFK) \wedge At(P_1, SFO) \wedge At(P_2, JFK) \\
&\quad \wedge Cargo(C_1) \wedge Cargo(C_2) \wedge Plane(P_1) \wedge Plane(P_2) \\
&\quad \wedge Airport(JFK) \wedge Airport(SFO)) \\
&Goal(At(C_1, JFK) \wedge At(C_2, SFO)) \\
&Action(Load(c, p, a), \\
&\quad \text{PRECOND}: At(c, a) \wedge At(p, a) \wedge Cargo(c) \wedge Plane(p) \wedge Airport(a) \\
&\quad \text{EFFECT}: \neg At(c, a) \wedge In(c, p)) \\
&Action(Unload(c, p, a), \\
&\quad \text{PRECOND}: In(c, p) \wedge At(p, a) \wedge Cargo(c) \wedge Plane(p) \wedge Airport(a) \\
&\quad \text{EFFECT}: At(c, a) \wedge \neg In(c, p)) \\
&Action(Fly(p, from, to), \\
&\quad \text{PRECOND}: At(p, from) \wedge Plane(p) \wedge Airport(from) \wedge Airport(to) \\
&\quad \text{EFFECT}: \neg At(p, from) \wedge At(p, to))
\end{aligned}
$$

ADL : Air Cargo Problem

Init(At(C1,SFO) ∩ At(C2,JFK) ∩ At(P1,SFO) ∩ At(P2, JFK) ∩ (P1: plane) ∩ (p2: plane) ∩ (C1: Cargo) ∩ (c2: Cargo) ∩ SFO:Airport ∩ JFK: Airport)

Goal(At(C1, JFK) ∩ At(C2,SFO))

Action((Load(c:cargo,p:plane,a:airport)
Precondition: At(c,a) ∩ at(p,a)
Effect: ~At(c,a) ∩ in(c,p))

Action(Fly(p : Plane, from : Airport, to : Airport),
PRECOND: At(p,from)
EFFECT : ~At(p,from) ∩ At(p,to))

Action(UnLoad(c:cargo,p:plane,a:airport)
Precondition: In(c,p) ∩ at(p,a)
Effect: At(c,a) ∩ ~in(c,p))

The simple spare tire problem

$Init(At(Flat, Axle) \wedge At(Spare, Trunk))$

$Goal(At(Spare, Axle))$

$Action(Remove(Spare, Trunk),$

PRECOND: $At(Spare, Trunk)$

EFFECT: $\neg At(Spare, Trunk) \wedge At(Spare, Ground))$

$Action(Remove(Flat, Axle),$

PRECOND: $At(Flat, Axle)$

EFFECT: $\neg At(Flat, Axle)$ A $At(Flat, Ground))$

$Action(PutOn(Spare, Axle),$

PRECOND: $At(Spare, Ground)$ A $\neg At(Flat, Axle)$

EFFECT: $\neg At(Spare, Ground) \wedge At(Spare, Axle))$

$Action(LeaveOvernight,$

PRECOND:

EFFECT: $\neg At(Spare, Ground)$ A $\neg At(Spare, Axle) \wedge \neg At(Spare, Trunk)$

A $\neg At(Flat, Ground)$ A $\neg At(Flat, Axle))$

Partial order planning

- Any planning algorithm that can place two actions into a plan without specifying which comes first is called a **partial-order planner**
- solution is represented as a graph of actions, not a sequence
- Dummy states- Start & Finish
- The partial-order solution corresponds to possible total-order plans; each of these is called a **linearization of the partial-order plan.**

What POP needs?

1. Set of actions
2. Set of ordering constraints
3. Set of causal links
4. Set of open preconditions

1. Set of actions

- Taken from
  - set of actions in the planning problem
  - "empty" plan contains just the Start and Finish actions
- Start state
  - no preconditions
  - Effect contains all the literals in the initial state
- Finish state
  - has no effects
  - Preconditions contain the goal literals of the planning problem.

2. Set of ordering constraints

- Each ordering constraint is of the form A « B, which is read as "A before B“
- Means that action A must be executed sometime before action B, but not necessarily immediately before.
- Any cycle-such as A « *B and B* « *A-represents a contradiction*, so an ordering constraint cannot be added to the plan if it creates a cycle.

3. Set of causal links

- **A causal link between two actions A and B in the plan is written** as
- $A \xrightarrow{p} B$ *and is read as "A* ***achieves p for B.****"*

- ***For example, the causal link***

RightSock $\xrightarrow{\text{RightsockOn}}$ *RightShoe*

  - asserts that *RightSockOn is an effect of the RightSock action and a precondition of RightShoe.*
  - *It also asserts that RightSockOn must remain true from the time of action RightSock to the time of action RightShoe.*
- *the plan may not be* extended by adding a new action *C that* ***conflicts with the causal link.***
- ***An action C*** conflicts with $A \longrightarrow B$ *if C has the effect ~p if C could (according to the ordering* constraints) come after A and before B.

4. Set of open preconditions

- A precondition is open if it is not achieved by some action in the plan.
- ordering constraints that put every other action after Start and before Finish):
  - Actions: { Rightsock, RightShoe, Leftsock, LeftShoe, Start, Finish}
  - Orderings:{RightSock « *RightShoe*, *Leftsock « LeftShoe}*
  - Links:{RightSock RightSockOn RightShoe,
    Leftsock *LeftSockk0n* → *LeftShoe*,
    RightShoe *RightShoeOn* → *Finish*,
    Leftshoe LeftShoe0n → *Finish}*
  - Open Preconditions:{} .

consistent plan

- **Definition-** a plan in which there are no cycles in the ordering constraints and no conflicts with the causal links.
- **solution -** A consistent plan with no open preconditions
- ***Total order plan- Every*** *linearization of a partial-order solution is a total-order solution whose execution from the initial state will reach a goal state.*
- A partial-order plan is executed by repeatedly choosing *any of the possible next actions.*
- The flexible ordering also makes it easier to combine smaller plans into larger ones, as each of the small plans can reorder its actions to avoid conflict with the other plans.

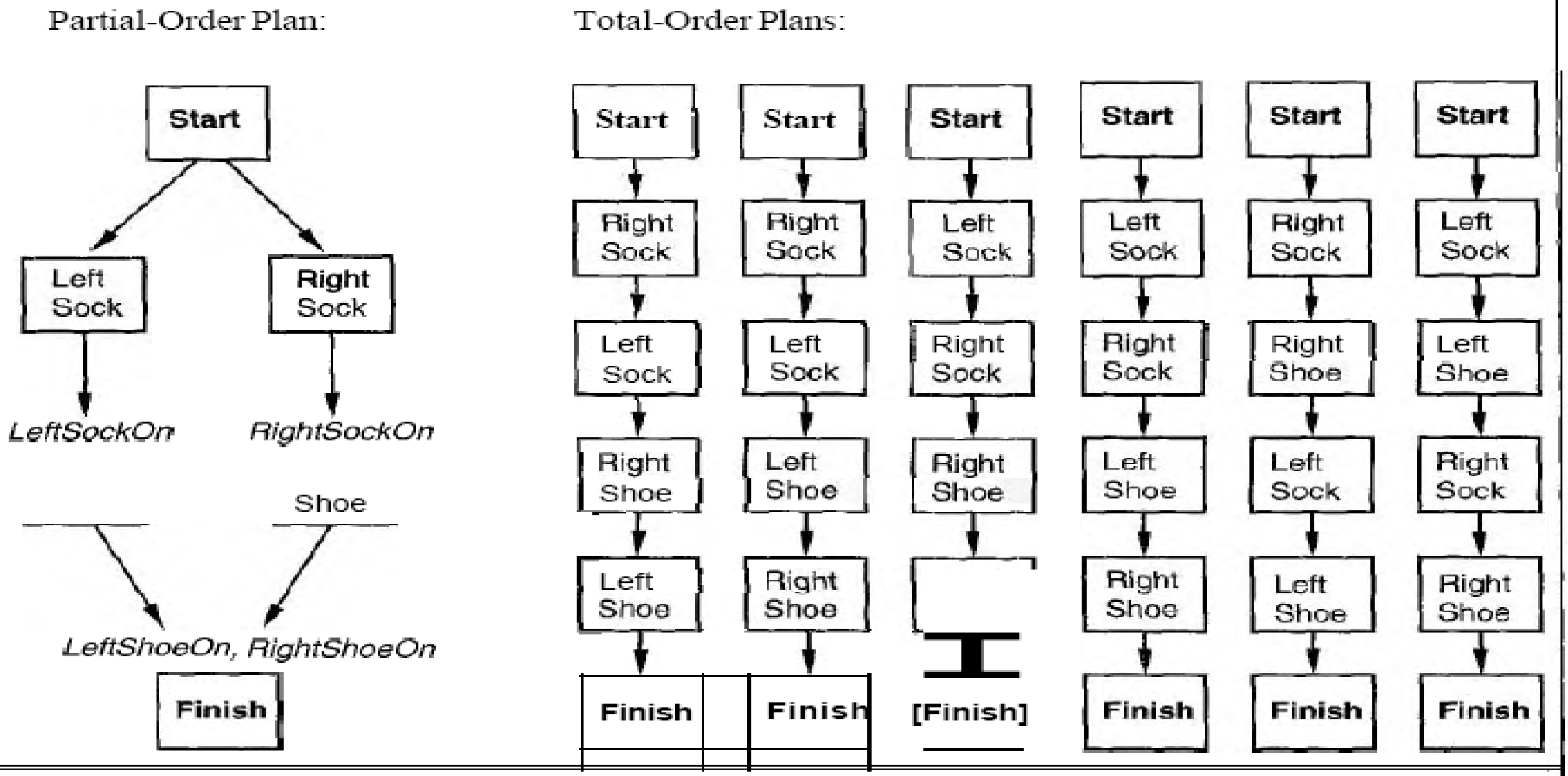


Figure 11.6 A partial-order plan for putting on shoes and socks, and the six corresponding linearizations into total-order plans.

PARTIAL ORDER PLANNING ALGORITHM

Step 1: The initial plan contains Start and Finish, the ordering constraint Start « Finish, and no causal links and has all the preconditions in Finish as open preconditions.

Step 2: The successor function arbitrarily picks one open precondition p on an action B and generates a successor plan for every possible consistent way of choosing an action A that achieves p.

Consistency is enforced as follows:

1. The causal link A P B and the ordering constraint A « *B are added to the plan.*

   Action A may be an existing action in the plan or a new one. If it is new, add it to the plan and also add Start « *A and A* « *Finish.*

2. We resolve conflicts between the new causal link and all existing actions and between the action A (if it is new) and all existing causal links.

   A conflict between A « *B and C is resolved by making C occur at some time outside the protection* interval, either by adding B « *C or C* « *A. We add successor states for either or* both if they result in consistent plans.

Step 3: The goal test checks whether a plan is a solution to the original planning problem. Because only consistent plans are generated, the goal test just needs to check that there are no open preconditions.

A POP example

$Init(At(Flat, Axle) \wedge At(Spare, Trunk))$

$Goal(At(Spare, Axle))$

$Action(Remove(Spare, Trunk),$

PRECOND: $At(Spare, Trunk)$

EFFECT: $\neg At(Spare, Trunk) \wedge At(Spare, Ground))$

$Action(Remove(Flat, Axle),$

PRECOND: $At(Flat, Axle)$

EFFECT: $\neg At(Flat, Axle) \wedge At(Flat, Ground))$

$Action(PutOn(Spare, Axle),$

PRECOND: $At(Spare, Ground) \wedge \neg At(Flat, Axle)$

EFFECT: $\neg At(Spare, Ground) \wedge At(Spare, Axle))$

$Action(LeaveOvernight,$

PRECOND:

EFFECT: $\neg At(Spare, Ground) \wedge \neg At(Spare, Axle) \wedge \neg At(Spare, Trunk)$
$\wedge \neg At(Flat, Ground) \wedge \neg At(Flat, Axle))$

Figure 11.7 The simple flat tire problem description.

POP solution

- Initialization:
  - Initial plan= { *Start*, *Finish}*
  - *Causal links={ }*
  - *Open preconditions={At(Spare,axle)}*
  - *Ordering constraints={Start «Finish }*
- Pick the only open precondition, *At(Spare, Axle) of Finish. Choose the only applicable* action, *PutOn(Spare, Axle).*
  - *Plan= { Start, Finish, PutOn(Spare,Axle)}*
  - *Causal links={PutOn(Spare,Axle) At(Spare,~~axle) Finish}~~*
  - *Open preconditions={At (Spare, Ground), ~At(flat,Axle)}*
  - *Ordering constraints={Start «Finish , Start « PutOn(Spare,Axle), PutOn(Spare,Axle) « Finish }*

- Pick the *At (Spare, Ground) precondition of PutOn(Spare, Axle). Choose the only* applicable action, *Remove(Spare,Trunk) to achieve it.*
  - *Plan= { Start, Finish, PutOn(Spare,Axle), Remove(Spare, Trunk),}*
  - *Causal links={PutOn(Spare,Axle) At(Spare,axle) Finish,*
    Remove(Spare,Trunk) At (Spare,Ground)PutOn(Spare,Axle) }
  - *Open preconditions={At(spare,trunk),~At(flat,Axle)}*
  - *Ordering constraints={Start «Finish , PutOn(Spare,Axle) « Finish, Remove(Spare,Trunk) « PutOn(Spare,Axle) }*

Remove(spare,trunk)

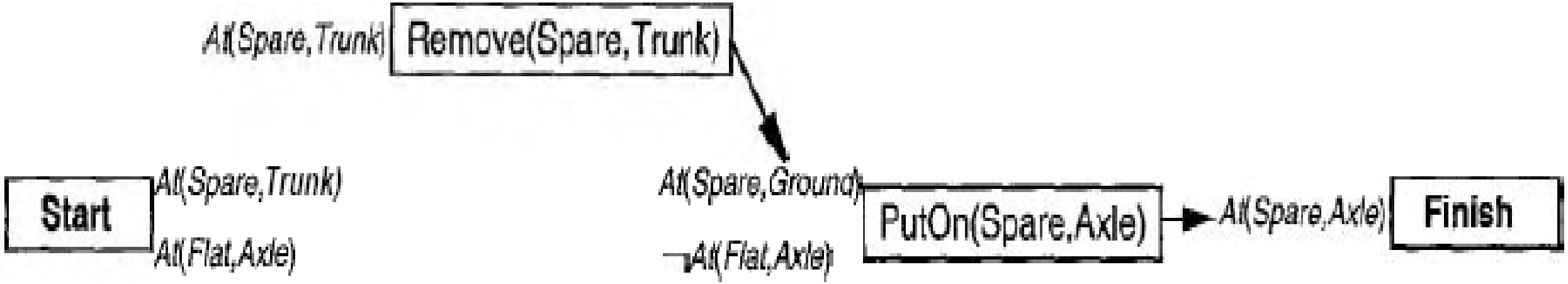


Figure 11.8 The incomplete partial-order plan for the tire problem, after choosing actions for the first two open preconditions. Boxes represent actions, with preconditions on the left and effects on the right. (Effects are omitted, except for that of the *Start* action.) Dark arrows represent causal links protecting the proposition at the head of the arrow.

- Pick the *~At(Flat, Axle) precondition of PutOn(Spare, Axle). If you choose LeaveOvernight action then it has the effect ~At(Spare, Ground), which means it conflicts* with the causal link

Remove(Spare,Trunk) At (Spare,Ground)PutOn(Spare,Axle)

- *So*, To resolve the conflict we add an ordering constraint putting *LeaveOvernight before Remove(Spare, Trunk).*

Ordering Conflicting actions

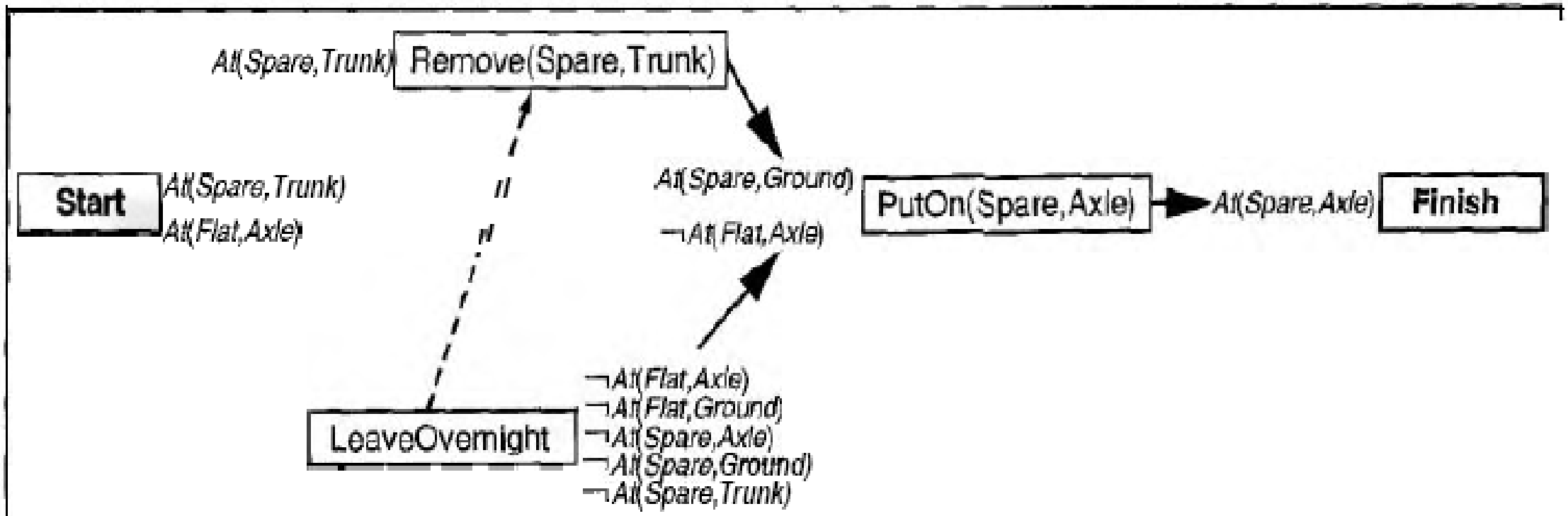


Figure 11.9 The plan after choosing *LeaveOvernight* as the action for achieving $\neg At(Flat, Axle)$. To avoid a conflict with the causal link from $Remove(Spare, Trunk)$ that protects $At(Spare, Ground)$, *LeaveOvernight* is constrained to occur before $Remove(Spare, Trunk)$, as shown by the dashed arrow.

- Pick the *Remove(Flat,Axle) precondition of ~At(Flat, Axle).*
  - *Plan= { Start,Finish,PutOn(Spare,Axle), Remove(Spare,Trunk), Remove(Flat,Axle)}*
  - *Causal links={PutOn(Spare,Axle) At(Spare,axle) Finish,*
    Remove(Spare,Trunk) At (Spare,Ground)PutOn(Spare,Axle)
    Remove(Flat,Axle) ~At(Flat,Axle) PutOn(Spare,Axle) }
  - *Open preconditions={At(spare,trunk),At(flat,Axle)}*
  - *Ordering constraints={Start«Finish,*
    PutOn(Spare,Axle)«Finish, Remove(Spare,Trunk)«PutOn(Spare,Axle),
    Remove(Flat,Axle))«PutOn(Spare,Axle)}

FinalPlan

- *Plan= { Start,Finish,PutOn(Spare,Axle), Remove(Spare,Trunk), Remove(Flat,Axle)}*
- *Causal links={PutOn(Spare,Axle) At(Spare,axle) Finish,*
  Remove(Spare,Trunk) At(Spare,Ground)P~~utOn(Spare,Axle)~~
  Remove(Flat,Axle) ~At(Flat,~~Axle) PutOn(Spare,Axle)~~
  start At(Flat,Axle) Remo~~ve(Flat,Axle)~~
  Start At(S~~pare,Trunk) Remove(~~Spare,Trunk)}
- *Open preco~~nditions={}~~*
- *Ordering constraints={Start«Finish,*
  PutOn(Spare,Axle)«Finish, Remove(Spare,Trunk)«PutOn(Spare,Axle),
  Remove(Flat,Axle))«PutOn(Spare,Axle)

Start «Remove(Spare,Trunk)
Start «Remove(Flat,Axle)}

Final plan

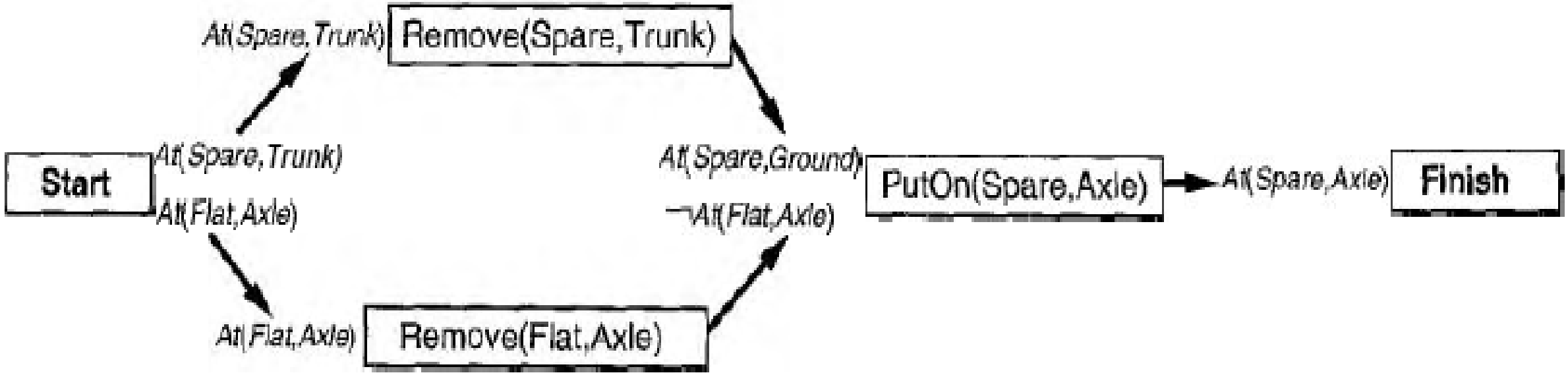
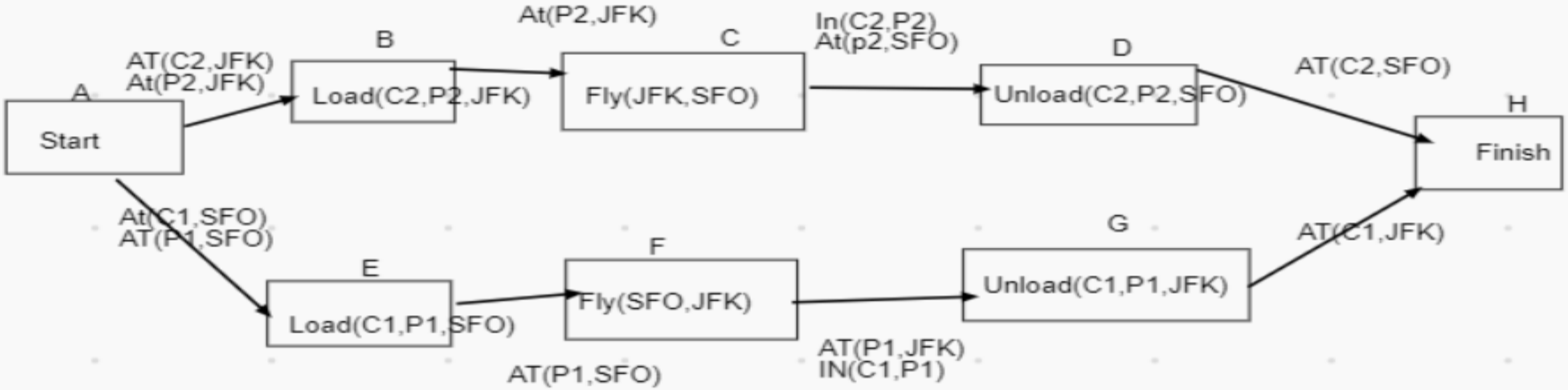


Figure 11.10 The final solution to the tire problem. Note that *Remove*(*Spare*, Trunk) and *Remove*(*Flat*, *Axle*) can be done in either order, as long as they are completed before the *Put*On(*Spare*,*Axle*) action.

Pop GRAPH FOR CARGO TRANSFER

TOP:
1. A-B-C-D-E-F-G-H
2. A-E-F-G-B-C-D-H
3. A-B-E-C-F-D-G-H
4. A-E-B-F-C-G-D-H
5. A-B-C-E-F-D-G-H
6. A-E-F-B-C-G-D-H
7. ?
8. ?
9. ?
10. ?

POP:



Applications

- Mobile robots
- Autonomous agents
  - NASA Deep Space planning agent
- Simulated environments
  - Goal-directed agents for training or games
- Web and grid environments
  - Composing queries or services

HIERCHICAL TASK PLANNING(HTN)

- Follows planning in real world approach
- Follows Hierarchy of actions concept
  - *major* action or *minor* action
  - Lower level activities would detail more precise steps for accomplishing the higher level tasks.

Planning for ”birthday party”

- Major Steps :
  - Invite friends
  - Decorations
  - Bake a cake
  - Party food, snacks and beverages
  - Cleaning after party
- Minor Steps :
  - Procuring cake ingridients
  - House cleaning before decorations
  - Take photos
  - Plan on assorted party food, cosidering food preferences of attendees
  - Arranging for sleepover if someone is staying back

General Property

- ***Postpone*** attempts to solve mere details, ***until*** major steps are in place.
- Higher level plan may run into difficulties at a lower level, causing the need to return to higher level again to produce appropriately ordered sequence.

Planner

- Identify a hierarchy of conditions
- Construct a plan in levels, postponing details to the next level
- Patch higher levels as details become visible

Example

- Actions required for “Travelling to Goa”:
  - Opening makemytrip.com (1)
  - Finding flight (2)
  - Buy Ticket (3)
  - Get taxi(2)
  - Reach airport(3)
  - Pay-driver(1)
  - Check in(1)
  - Boarding plane(2)
  - Reach Goa(3)

Example

- 1st level Plan :
  - Buy Ticket (3), Reach airport(3), Reach Goa(3)
- 2nd level Plan :
  - Finding flight (2), Buy Ticket (3), Get taxi(2), Reach airport(3), Boarding plane(2), Reach Goa(3)
- 3rd level Plan (final) :
  - Opening makemytrip.com (1), Finding flight (2), Buy Ticket (3), Get taxi(2), Reach airport(3), Pay-driver(1), Check in(1), Boarding plane(2), Reach Goa(3)

Hierarchical Task Network

- Hierarchy abstraction achieved through *methods*.
- A method is a pair (α, d) , where
  - α is the non-primitive task, and
  - d is the task network to achieve the task α

Top-down synergy

- Plans at higher level of hierarchy achieves more effects than at a lower level.
- A part of the plan can be pruned if its post-conditions do not overlap with any other plan's post-condition.

Example: Build a house

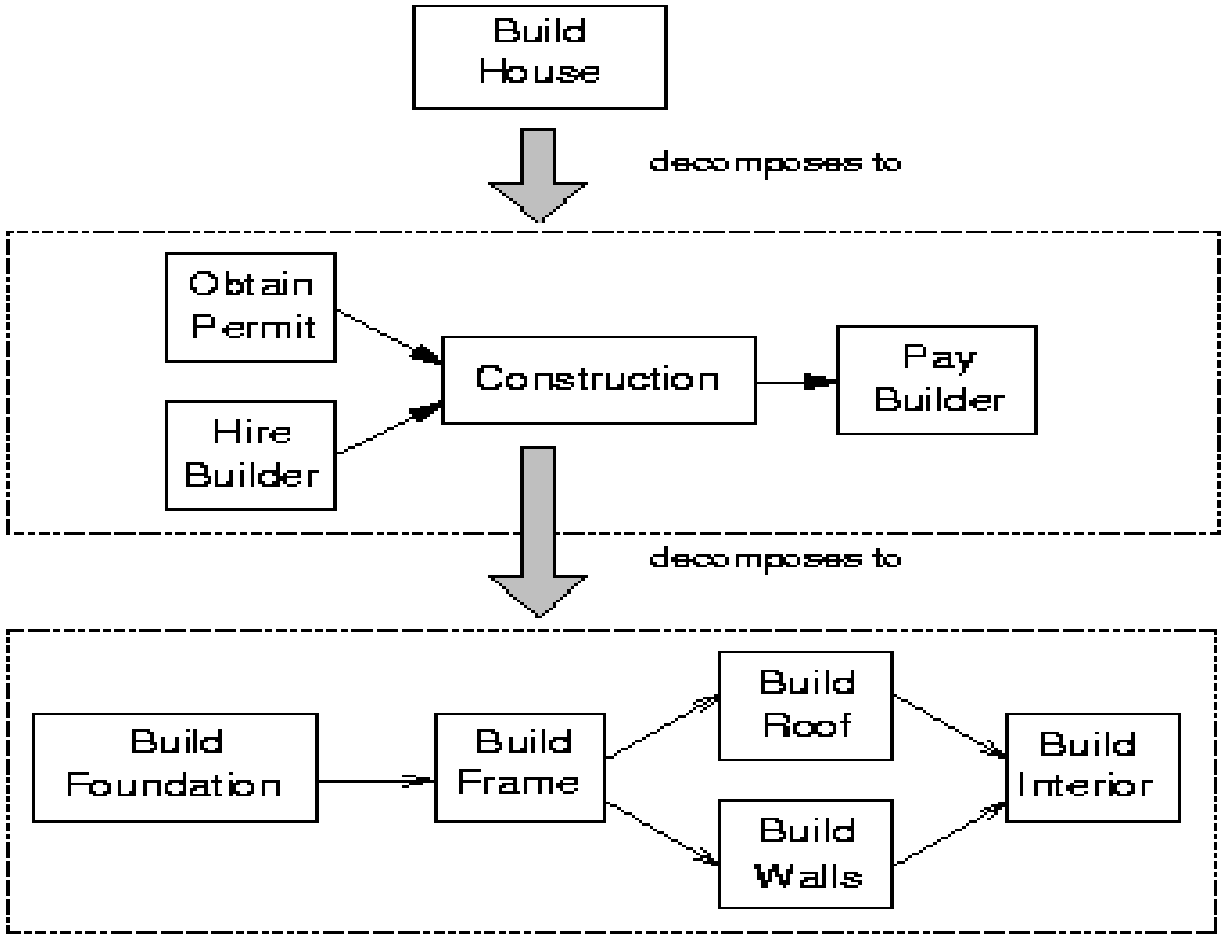
$Action(BuyLand, \text{PRECOND}: Money, \text{EFFECT}: Land \wedge \neg Money)$
$Action(GetLoan, \text{PRECOND}: GoodCredit, \text{EFFECT}: Money \wedge Mortgage)$
$Action(BuildHouse, \text{PRECOND}: Land, \text{EFFECT}: House)$

$Action(GetPermit, \text{PRECOND}: Land, \text{EFFECT}: Permit)$
$Action(HireBuilder, \text{EFFECT}: Contract)$
$Action(Construction, \text{PRECOND}: Permit \wedge Contract,$
$\quad \text{EFFECT}: HouseBuilt \wedge \neg Permit)$
$Action(PayBuilder, \text{PRECOND}: Money \wedge HouseBuilt,$
$\quad \text{EFFECT}: \neg Money \wedge House \wedge \neg Contract)$

$Decompose(BuildHouse,$
$\quad Plan(\text{STEPS}: \{S_1 : GetPermit, S_2 : HireBuilder,$
$\qquad S_3 : Construction, S_4 : PayBuilder\}$
$\qquad \text{ORDERINGS}: \{Start \prec S_1 \prec S_3 \prec S_4 \prec Finish, \quad Start \prec S_2 \prec S_3\},$
$\qquad \text{LINKS}: \{Start \xrightarrow{Land} S_1, Start \xrightarrow{Money} S_4,$
$\qquad S_1 \xrightarrow{Permit} S_3, S_2 \xrightarrow{Contract} S_3, S_3 \xrightarrow{HouseBuilt} S_4,$
$\qquad S_4 \xrightarrow{House} Finish, S_4 \xrightarrow{\neg Money} Finish\}))$

Figure 12.6 Action descriptions for the house-building problem and a detailed decomposition for the *BuildHouse* action. The descriptions adopt a simplified view of money and an optimistic view of builders.

HTN planning: example



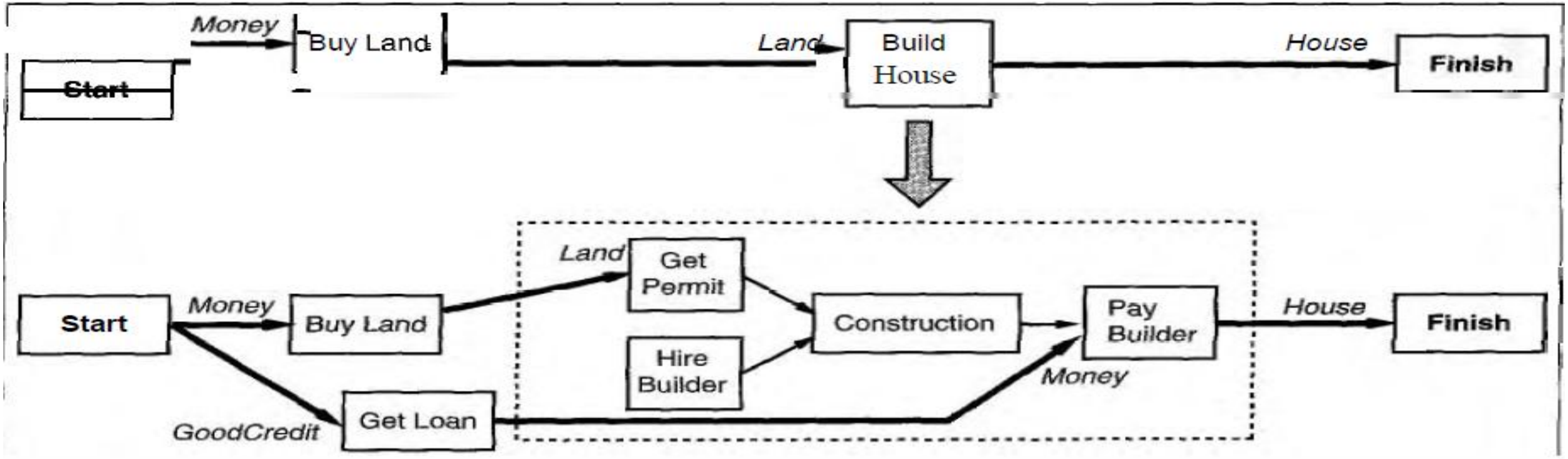
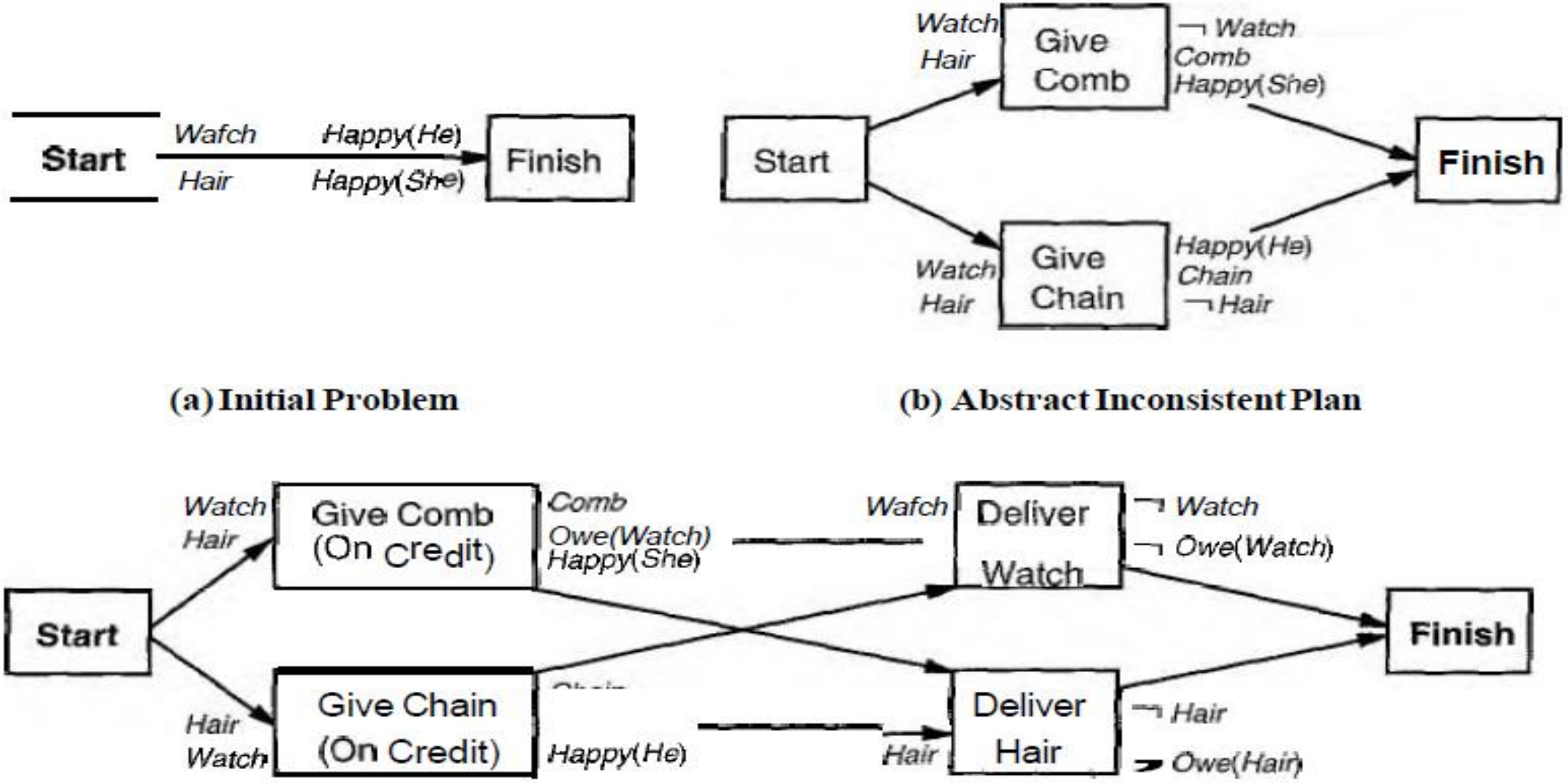


Figure 12.7 Decomposition of a high-level action within an existing plan. The *BuildHouse* action is replaced by the decomposition from Figure 12.5. The external precondition *Land* is supplied *by* the existing causal link from *BuyLand*. The external precondition *Money* remains open after the decomposition step, so we add a new action, *GetLoan*.

Gift of Maggi by o'henry



(a) Initial Problem

(b) Abstract Inconsistent Plan

(c) Decomposition of (b) into a Consistent Solution

Conclusion

- For complex problems hierarchical planning is much more efficient than single level planning.
- Improves performance as number of operator in the problem increases.
- HTN planning gives more expressivity
- Merging opens door to accomplish a complete plan from incomplete individual plans
- Integration with ontology opens door for automatic planning
- Reduces man machine gap.

reference

- “AIMA”, Russel & Norving, Pearson education
- Lecture slides, Prof. Sarkar, IIT KGP